VSS: Visualization of Search and Sort Algorithms

Arya Acharya
Department of Computer Science and Engineering
CHRIST (Deemed to be University)
Email: aryaacharya2500@gmail.com

***Abstract*—This project outlines the design and implementation of an interactive application created on the Android operating system, aimed at providing a visual representation of basic algorithms related to sorting and searching data. The application demonstrates Radix Sort, Merge Sort, Tree Sort, Linear Search, Jump Search, and Tree Search algorithms in a visual format, where bars are used to represent the algorithms' internal operations. The application is created to provide a visual representation of algorithms, transforming abstract procedures into a more comprehensible and interactive format.**

The application is equipped with a real-time performance analysis feature, where the execution time is shown in milliseconds, memory usage in kilobytes, and the total number of operations performed during execution. The application is further equipped with a frequency-based sound mapping feature, where numerical values are related to specific levels of pitches, providing a sense of the algorithms' transition. Moreover, a Text-to-Speech feature is included, providing a detailed explanation of each algorithm, making the application more interactive and comprehensible.

The system was implemented using Unity Engine with C# as the primary programming language, utilizing the DOTween animation library to facilitate smooth and controlled transitions. The application architecture prioritizes modularity, responsiveness, and visualization. The application combines graphical animation, performance metrics, and auditory cues within a mobile application to create an engaging educational tool that enhances theoretical and practical knowledge regarding algorithms.

***Index Terms*—Algorithm Visualization, Android, Sorting Algorithms, Searching Algorithms, Unity Engine, Educational Technology, Interactive Learning**

I. Introduction

Algorithms are the intellectual foundation of computer science. Every computer system, whether it is as simple as a calculator or as complex as the distributed infrastructure of the cloud, relies on the use of well-defined procedures that can process information in an efficient manner. These procedures, called algorithms, dictate the way information is structured, searched, transformed, and optimized.

Among all the different categories of algorithms that can be learned in an undergraduate curriculum, sorting and searching algorithms have intrinsic importance as they represent the basic mechanism for data transformation.

Sorting algorithms are used for arranging data in any specific order, either ascending or descending. Searching algorithms are used for searching any specific data from the given data set. These two algorithms form the basic backbone of many different systems that can be developed.

The study of algorithms has traditionally been concerned with the analysis of time complexity. Students are able to understand how to classify algorithms according to time complexity with the help of Big O notation. For instance, Merge Sort has an $O(n \log n)$ time complexity, Linear Search has an $O(n)$ time complexity, and Jump Search has a time complexity of $O(\sqrt{n})$. Although it gives important theoretical knowledge, it may still be at an abstract level unless learners see how it works in practice.

Visualization has tremendous potential to bridge the gap between theoretical knowledge and implementation knowledge. Students can see elements moving around and associate them with movement patterns.

Moreover, learning aids can be effective in retaining knowledge. Mobile technology can be used as an important medium to carry out educational applications. Android is one of the most popular mobile device platforms in use today. Its availability is unparalleled compared to visualization tools.

The project leverages the deployment strength of the Android platform to implement a portable algorithm visualization tool.

Several motivating factors influenced the development of this project.

Students face difficulties in understanding element comparison in sorting algorithms, recursion in divide-and-conquer techniques, tree-based insertion and traversal techniques, and complexity in execution steps. The difficulties stem from the dynamic execution of algorithms in comparison to a static environment.

The majority of visualizers for algorithms concentrate on graphical representation alone. However, research on cognition has shown that multi-sensory inputs lead to higher memory retention. The inclusion of auditory feedback could increase pattern recognition capabilities.

Though many visualization tools exist that can animate algorithms, they do not always include real-time execution metrics such as execution time in milliseconds, memory usage, and step count. This visualization provides a tangible representation of the effects of theoretical complexity analysis.

The Android application allows offline usage and offers a better user interface than web-based visualizers that require browsers and internet connectivity.

Through academic observation and informal student feedback, several issues were identified. Students memorize algorithm steps without understanding the movement of data. Recursive algorithms such as Merge Sort are conceptually difficult to visualize. Tree-based algorithms are abstract without

dynamic structural representation. Time complexity analysis remains theoretical without runtime comparison. Most learning tools are not optimized for mobile platforms.

These challenges collectively highlight the need for a structured, interactive, and portable visualization platform.

The problem addressed in this work involves creating an Android application to visually present and interactively demonstrate the internal execution process of Radix Sort, Merge Sort, Tree Sort, Linear Search, Jump Search, and Tree Search.

The primary objectives include implementing six search and sort algorithms, visually animating algorithm execution, updating execution time, memory usage, and step count in real time, and generating random data for experimentation.

Secondary objectives include integrating the DOTween animation engine, incorporating Text-to-Speech for narration, mapping numeric values to frequency ranges for auditory feedback, and developing a responsive Android user interface.

The scope of the project involves educational visualization of moderate-sized arrays, performance comparison between algorithms, mobile deployment using Unity, and structured user interface navigation between modules.

II. Literature Review

Algorithms form one of the integral parts of computer science education. However, students have been found to face difficulties in comprehending the process of algorithm execution through text or pseudocode descriptions. Educational research has been conducted to improve learning outcomes through algorithm visualization techniques. Algorithm visualization systems graphically represent algorithmic operations such as swap operations, recursive operations, and data movement operations. The visualization environment enables students to view the algorithmic process step by step and relate it to theoretical complexity analysis. Visualization improves students' conceptual understanding when used in conjunction with active interaction rather than passive viewing of animations [1].

The literature on algorithm education can be broadly categorized into three areas: visualization techniques for algorithm education, studies on sorting and searching algorithm analysis, and the development of interactive educational tools using modern computing platforms.

A. Algorithm Visualization

In traditional educational settings, algorithms are usually explained through lectures, textbook reading, and manual tracing of pseudocode. Students are expected to analyze algorithm behavior by mentally simulating operations described by code or mathematical expressions. This approach helps develop theoretical thinking but may not provide sufficient visual cues regarding how data elements change during execution.

Desktop-based visualization systems were introduced to overcome these limitations by providing graphical representations of arrays, trees, and algorithm operations. Students can visually understand sorting and searching algorithms, adjust execution speed, and interactively observe algorithm behavior. Studies suggest that visualization tools are effective when students actively interact with the simulation environment rather than only observing animations [1].

With the advancement of internet technologies, algorithm visualization tools were later introduced as web-based systems. These systems improve accessibility through web browsers but may experience performance limitations and often lack sufficient analytical metrics regarding algorithm execution.

The rapid growth of mobile computing has created new opportunities for educational applications. Interactive visualization on mobile devices improves learner engagement and provides better access to educational resources [2]. However, many available mobile applications support only a limited number of algorithms and lack integrated performance analytics.

B. Sorting Algorithms

Radix Sort is commonly presented as a non-comparative sorting algorithm. It sorts data according to individual digits rather than directly comparing values. Visualization effectively illustrates how elements move through intermediate buckets during multiple passes.

Merge Sort is one of the most widely studied divide-and-conquer algorithms. The algorithm divides an array into smaller segments, recursively sorts each segment, and merges them into a final sorted structure. Interactive visualization allows students to observe recursive division and merging operations while understanding the algorithm's time complexity of $O(n \log n)$.

Tree Sort demonstrates how data structures influence algorithm performance. Elements are inserted into a Binary Search Tree before being retrieved through inorder traversal. Visualization of the tree structure assists students in understanding hierarchical relationships and illustrates how unbalanced trees reduce algorithm efficiency.

C. Searching Algorithms

Linear Search compares each element sequentially until the required element is found. Although inefficient for large datasets, it provides a useful reference for comparing more advanced searching algorithms.

Jump Search improves searching efficiency in sorted arrays by jumping through fixed intervals before performing linear search within the identified block. Visualization clearly demonstrates the block-based searching process.

Tree Search uses Binary Search Tree traversal to locate elements efficiently. When the Binary Search Tree remains balanced, logarithmic search performance can be achieved. Visualization assists learners in understanding traversal decisions while searching through tree structures.

D. Performance Metrics and Educational Support

Modern visualization systems increasingly integrate execution metrics including execution time, memory usage, and operation count. Performance analytics help relate theoretical complexity analysis with practical execution behavior [3].

Animation further enhances algorithm understanding by illustrating comparisons, swaps, and structural modifications. Interactive visualization environments where learners can control execution speed have been shown to improve learning outcomes [4].

Educational research also incorporates multi-sensory learning strategies. Audio feedback represents algorithm behavior through sound patterns by mapping numerical values to audio frequencies. Text-to-Speech technology further improves understanding by explaining algorithm logic during execution.

E. Research Gap

Although numerous visualization tools exist for algorithm education, several research gaps remain. Many visualization systems focus exclusively on either sorting algorithms or searching algorithms rather than integrating both within a single environment. Existing tools frequently emphasize graphical animation while providing limited real-time performance metrics.

Few educational applications integrate animation, execution metrics, auditory feedback, and Text-to-Speech within a unified mobile learning environment. Mobile-based algorithm visualization platforms supporting multiple visualization techniques therefore remain limited.

TABLE I
SUMMARY OF LITERATURE SURVEY

Reference	Key Contribution
Fouh et al. (2016)	Visualization improves comprehension through active interaction.
McInnes et al. (2018)	Visualization improves interpretation of complex computational processes.
Wang et al. (2020)	Machine learning enhances visualization systems.
Mastorodimos et al. (2020)	Visualization improves computational thinking and self-efficacy.
S. Šimonák (2020)	Visualization increases engagement in algorithms and data structures education.
Zavgorodniaia (2021)	Interactive visualization improves learner engagement.
Paredes-Velasco et al. (2023)	Multi-modal visualization enhances learning experiences.
Shi (2024)	Visualization is widely adopted in online educational platforms.
I. Rent¸ea (2025)	Interactive visualizations improve student performance and learning outcomes.
J. Liu (2025)	Reviews recent approaches and trends in teaching algorithm design.

Table I summarizes the major contributions identified in the reviewed literature.

III. PROPOSED METHODOLOGY

The research methodology incorporates software engineering and educational technology to design, implement, and evaluate the proposed Android-based visualization system. The methodology involves requirement analysis, system architecture design, algorithm implementation, visualization layer integration, metrics measurement integration, testing, validation, and performance analysis.

The project follows a design-oriented research model consisting of the following stages:

1) Requirement Analysis
2) System Architecture Design
3) Algorithm Implementation
4) Visualization Layer Integration
5) Metrics Measurement Integration
6) Testing and Validation
7) Performance Analysis

Each stage contributes incrementally to the development of the final application.

A. System Requirements

The functional requirements include generating random integer arrays, executing six algorithms, animating comparisons and swaps, displaying real-time execution metrics, mapping numeric values to audio frequencies, and providing Text-to-Speech explanations.

The non-functional requirements include smooth animation transitions, a responsive user interface, low memory footprint, offline accessibility, and a modular and maintainable architecture.

B. System Architecture

The application is structured using a modular architecture consisting of the following components:

- Main Menu Controller
- Sorting Visualization Module
- Searching Visualization Module
- Metrics Manager
- Sound Manager
- Text-to-Speech Manager

Each module operates independently and communicates through structured interfaces. Unity Engine was selected due to its cross-platform deployment capability and strong user interface support. The development environment includes Unity Engine, C# Programming Language, DOTween Animation Library, Android SDK, and Visual Studio Code.

C. Algorithm Execution

Each algorithm is implemented as an individual coroutine to allow stepwise animation during execution.

The execution flow consists of the following stages:

1) User initiates the algorithm.
2) Stopwatch is started.
3) Algorithm is executed step-by-step.
4) Animation is applied.
5) Performance metrics are updated dynamically.
6) Stopwatch is stopped.

The modular architecture separates visualization logic from algorithm computation logic. Coroutines control the animation flow while a stopwatch measures runtime performance. Step counting is incremented during comparison and swapping operations. Numerical values are proportionally mapped to sound frequencies.

```
IEnumerator Merge(int left, int mid, int right)
{
    int n1 = mid - left + 1;
    int n2 = right - mid;

    int[] leftArray = new int[n1];
    int[] rightArray = new int[n2];

    for (int i = 0; i < n1; i++) leftArray[i] = data[left + i];
    for (int j = 0; j < n2; j++) rightArray[j] = data[mid + 1 + j];

    int k = left;
    int iIdx = 0, jIdx = 0;

    while (iIdx < n1 && jIdx < n2)
    {
        HighlightBar(left + iIdx, Color.yellow);
        HighlightBar(mid + 1 + jIdx, Color.yellow);
        AlgorithmAudioGenerator.Instance.PlayPing(data[left + iIdx], 100);

        yield return new WaitForSeconds(0.2f);

        if (leftArray[iIdx] <= rightArray[jIdx])
        {
            data[k] = leftArray[iIdx];
            UpdateBarVisual(k, data[k]);
            iIdx++;
        }
        else
        {
            data[k] = rightArray[jIdx];
            UpdateBarVisual(k, data[k]);
            jIdx++;
        }
        k++;
    }

    while (iIdx < n1)
    {
        data[k] = leftArray[iIdx];
        UpdateBarVisual(k, data[k]);
        iIdx++; k++;
        yield return new WaitForSeconds(0.1f);
    }

    while (jIdx < n2)
    {
        data[k] = rightArray[jIdx];
        UpdateBarVisual(k, data[k]);
        jIdx++; k++;
        yield return new WaitForSeconds(0.1f);
    }
```

Fig. 1. Implementation of the Merge Sort visualizer script.

D. Implementation Modules

The sorting module implements Radix Sort, Merge Sort, and Tree Sort. Each algorithm is developed using structured C# scripts integrated within Unity.

Radix Sort determines the maximum value, performs counting sort for each digit, and animates each rearrangement.

Merge Sort recursively divides the array into smaller segments before merging them into the final sorted structure. Recursive depth is visualized through progressive animation.

Tree Sort inserts elements into a Binary Search Tree and performs inorder traversal to generate sorted output.

The searching module implements Linear Search, Jump Search, and Tree Search.

Linear Search sequentially compares elements while highlighting every comparison step.

Jump Search performs block jumps followed by linear search within the identified segment. The step size is calculated using the square root of the array size.

Tree Search performs recursive Binary Search Tree traversal for efficient searching.

```
IEnumerator JumpSearchCoroutine(int target)
{

    System.Array.Sort(data);
    RegenerateSortedVisuals();

    yield return new WaitForSeconds(0.5f);

    AlgorithmMetrics.Instance.StartTracking(data.Length);

    int n = data.Length;
    int step = Mathf.FloorToInt(Mathf.Sqrt(n));
    int prev = 0;
    float maxVal = 80f;

    while (prev < n && data[Mathf.Min(step, n) - 1] < target)
    {
        AlgorithmMetrics.Instance.AddStep();

        int checkIndex = Mathf.Min(step, n) - 1;

        Highlight(checkIndex, Color.cyan);
        AlgorithmAudioGenerator.Instance.PlayPing(data[checkIndex], maxVal);

        yield return new WaitForSeconds(animationDelay);

        ResetBar(checkIndex);

        prev = step;
        step += Mathf.FloorToInt(Mathf.Sqrt(n));

        if (prev >= n)
        {
            AlgorithmMetrics.Instance.StopTracking();
            yield break;
        }
    }
```

Fig. 2. Implementation of the Jump Search visualizer script.

```
public void AddStep()
{
    stepCount++;
    UpdateUI();
}

10 references
public void StopTracking()
{
    stopwatch.Stop();
    UpdateUI();
}

3 references
private void UpdateUI()
{
    if (metricsText == null) return;

    float elapsedMilliseconds = stopwatch.ElapsedMilliseconds;
    float estimatedMemoryKB = baseMemory / 1024f;

    metricsText.text =
        "Real-Time Metrics\n" +
        "Time: " + elapsedMilliseconds + " ms\n" +
        "Space: " + estimatedMemoryKB.ToString("F2") + " KB\n" +
        "Steps: " + stepCount;
}
```

Fig. 3. Implementation of algorithm metrics tracking.

E. Performance Metrics and Audio Mapping

The visualization system displays execution time in milliseconds, memory usage in kilobytes, and step count during algorithm execution. Runtime performance is measured using stopwatch functionality implemented in C#, while memory usage is estimated through Unity profiling.

Linear mapping converts numerical values into corresponding frequency ranges. Lower values produce lower-pitch tones, whereas higher values generate higher-frequency audio signals

```
void OnAudioFilterRead(float[] data, int channels)
{
    for (int i = 0; i < data.Length; i += channels)
    {
        float waveValue = 0;

        if (isPlaying && sampleCount < stopSample)
        {
            phase += 2 * Mathf.PI * frequency / sampleRate;
            waveValue = Mathf.Sin((float)phase);

            float remainingSamples = stopSample - sampleCount;
            float envelope = Mathf.Clamp01(remainingSamples / (waveDuration * sampleRate));
            waveValue *= envelope * 0.2f;
        }
        else if (sampleCount >= stopSample)
        {
            isPlaying = false;
        }

        for (int j = 0; j < channels; j++)
        {
            data[i + j] = waveValue;
        }

        sampleCount++;

        if (phase > 2 * Mathf.PI) phase -= 2 * Mathf.PI;
    }
}
```

Fig. 4. Implementation of the algorithm audio generator.

using proportional scaling.

The Text-to-Speech module provides explanations of algorithm logic when users select the information button. The module is implemented using the Android native Text-to-Speech engine through the Unity Android bridge interface.

F. Testing

Testing consisted of unit testing, integration testing, performance testing, and informal user testing.

Unit testing verified each algorithm independently using predefined datasets.

Integration testing verified synchronization between the user interface and visualization components.

Performance testing evaluated execution time using the stopwatch mechanism.

Informal user testing was conducted to obtain feedback regarding usability and visualization quality.

IV. Results and Discussion

The implemented application was evaluated by executing the search and sort algorithms using randomly generated data. The evaluation considered execution time, memory usage, and step count during visualization.

A. Sorting Algorithms

The sorting algorithms were tested on arrays of moderate size to provide clear visualization on mobile devices.

Figure 5 illustrates the execution of Radix Sort. Stable execution behavior, linear increase in execution time relative to the number of digits, and smooth visualization transitions were observed.

Figure 6 shows Merge Sort execution. Consistent performance was observed together with higher memory usage because of auxiliary arrays. The recursive division of the array was clearly visualized.

Tree Sort performance depended on insertion order. The number of execution steps increased when the Binary Search

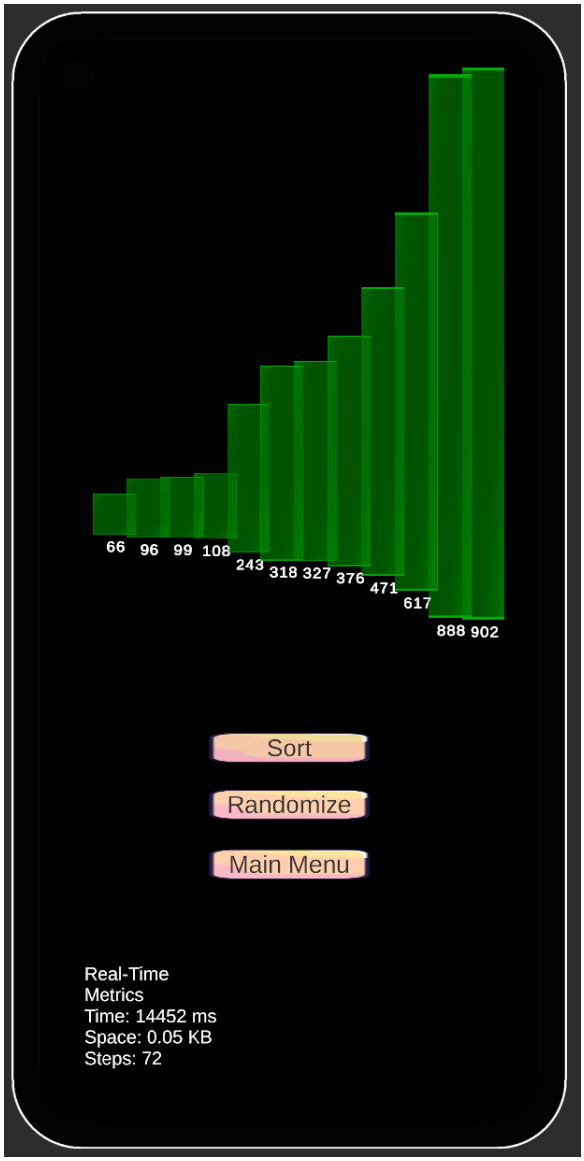


Fig. 5. Radix Sort execution output.

Tree became skewed. The visualization effectively demonstrated Binary Search Tree behavior.

B. Searching Algorithms

Linear Search performed sequential scanning of array elements. The step count remained proportional to the target position, and execution time increased when the element was located near the end of the array.

Jump Search reduced the number of comparisons compared with Linear Search and demonstrated improved performance for larger datasets.

Tree Search demonstrated logarithmic traversal behavior when the Binary Search Tree remained balanced. Performance varied according to the balance of the tree.

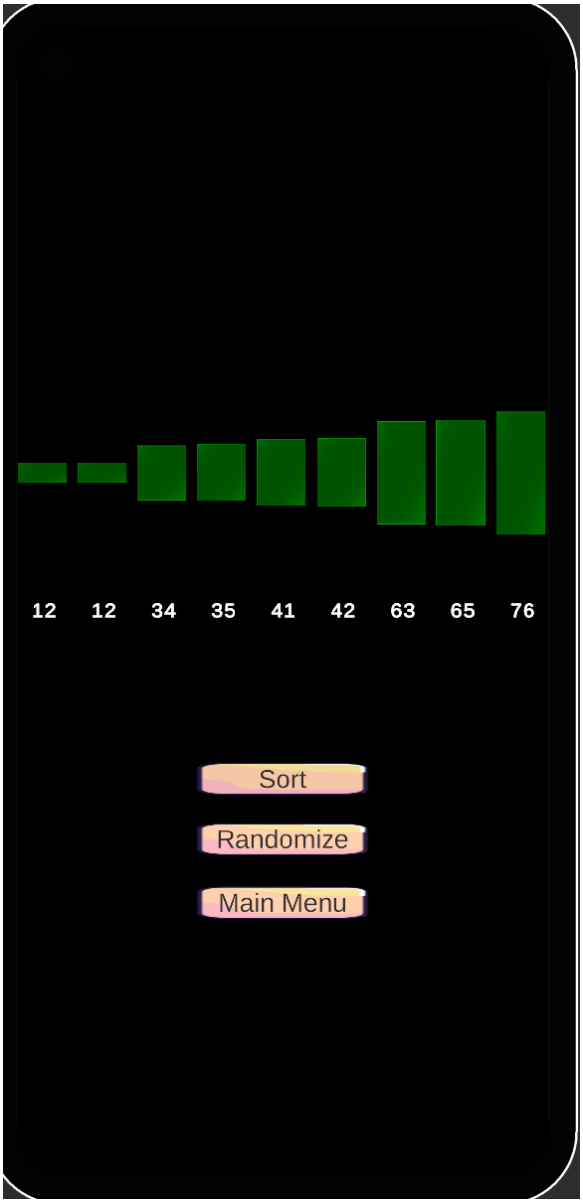


Fig. 6. Merge Sort execution output.

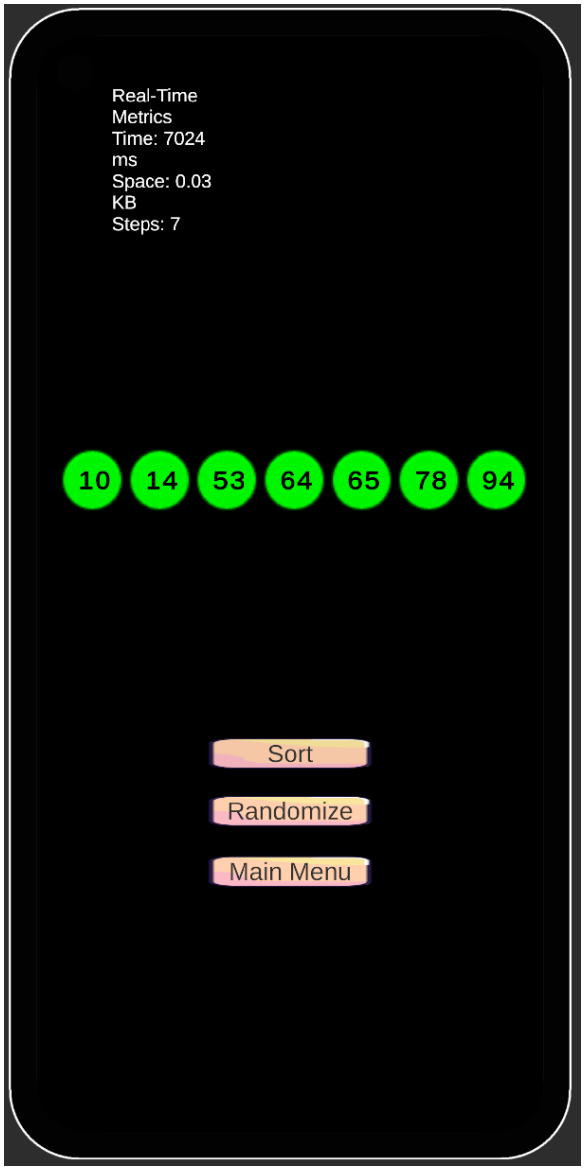


Fig. 7. Tree Sort execution output.

C. Performance Analysis

Execution times followed the expected theoretical complexity trends.

Linear Search showed linear growth.

Jump Search reduced the number of comparisons significantly.

Merge Sort maintained stable logarithmic growth.

Radix Sort demonstrated near-linear execution behavior.

Memory usage was highest for Merge Sort because of auxiliary space requirements.

Table II summarizes the observed algorithm performance.

The visualization plays an important role in improving algorithm understanding. Animation enables learners to observe element movement patterns during execution.

TABLE II
COMPARATIVE ANALYSIS OF IMPLEMENTED ALGORITHMS

Algorithm	Time Complexity	Observed Performance
Radix Sort	$O(nk)$	Stable and consistent
Merge Sort	$O(n \log n)$	Consistent with higher memory usage
Tree Sort	$O(n \log n)$ (average)	Depends on tree balance
Linear Search	$O(n)$	Slow for larger datasets
Jump Search	$O(\sqrt{n})$	Faster than Linear Search
Tree Search	$O(\log n)$ (average)	Efficient when balanced

The integration of sound mapping increases learner engagement by associating higher numerical values with higher-frequency sounds. Text-to-Speech provides additional explanation of algorithm logic, while execution metrics relate theoretical complexity analysis with practical algorithm behavior.

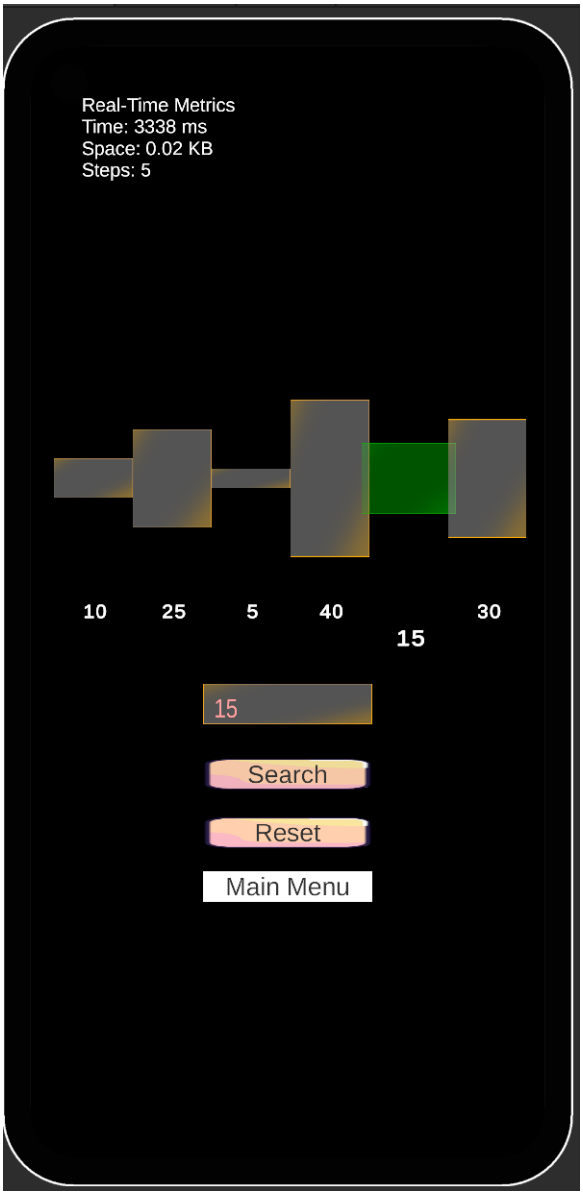


Fig. 8. Linear Search execution.

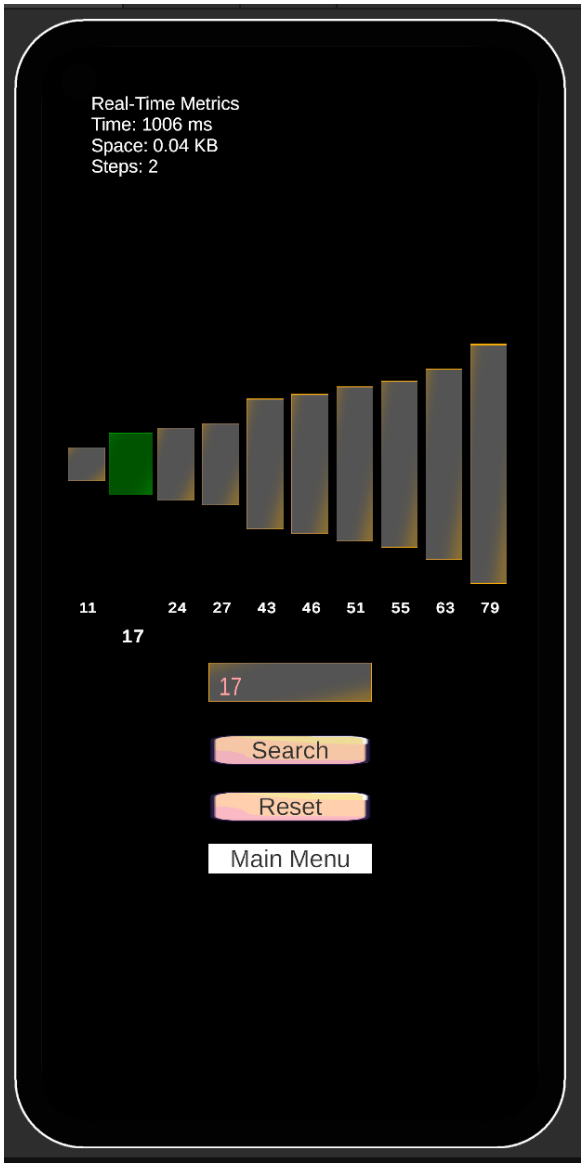


Fig. 9. Jump Search execution.

V. Conclusion

VSS: Visualization of Search and Sort Algorithms illustrates Radix Sort, Merge Sort, Tree Sort, Linear Search, Jump Search, and Tree Search using interactive visualization, performance statistics, sound feedback, and Text-to-Speech.

The application bridges the gap between theoretical algorithm analysis and practical visualization. DOTween enables smooth animation, real-time metrics improve analytical understanding, sound mapping supports multi-sensory learning, and Text-to-Speech provides algorithm explanations.

The proposed system satisfies the project objectives and contributes an interactive mobile platform for algorithm education.

VI. Future Work

Future enhancements may include the addition of algorithms such as Quick Sort and Heap Sort, implementation of shuffling algorithms, support for user-defined input arrays, interactive learning modules with quizzes, graphical visualization of Tree Sort and Tree Search, performance benchmarking tools, and deployment on the iOS platform.

References

[1] E. Fouh, M. Akbar, and C. Shaffer, "The Role of Visualization in Computer Science Education," ACM Transactions on Computing Education, vol. 16, no. 2, 2016.

[2] M. Paredes-Velasco, J. R. García, and M. González, "Augmented Reality with Algorithm Animation for Enhancing Student Learning," Multimedia Tools and Applications, 2023.

[3] D. Mastorodimos, A. Karampelas, and P. Kourouthanassis, "Algorithm Visualization and Its Impact on Self-Efficacy and Computational Thinking," 2020.

[4] A. Zavgorodniaia, “Algorithm Visualization and the Elusive Modality Effect,” Proceedings of the ACM Conference on Innovation and Technology in Computer Science Education, 2021.
[5] J. Shi, “Visualization Analysis of Online Education Systems,” 2024.
[6] Q. Wang, W. Chen, and Y. Wu, “A Survey on Machine Learning for Visualization,” arXiv preprint arXiv:2012.00467, 2020.
[7] L. McInnes, J. Healy, and J. Melville, “UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction,” arXiv preprint arXiv:1802.03426, 2018.
[8] I. Rent¸ea, “Interactive Visualizations and Their Effect on Student Performance,” Proceedings of the ACM Conference on Learning Technologies, 2025.
[9] J. Liu, “Teaching Algorithm Design: A Literature Review,” ACM Transactions on Computing Education, 2025.
[10] S. Šimonák, “Increasing Engagement in Algorithms and Data Structures Education Using Visualization,” Informatica Journal, vol. 44, no. 2, 2020.